

文档三 · 日志报错解读手册

打印设置：A4、边距默认。用法：这是"查字典"式的手册——遇到报错翻到对应条目；A组前10条建议熟读到条件反射。

第 1 页 · 理念篇：日志的构造与读法

1. 日志是什么？

程序运行时会给自己写"工作日记"，这就是日志（log）。程序死前留下的遗言，就是报错信息——它其实是程序在告诉你："我为什么干不下去了，你去看看什么原因。"报错不是敌人，是线索。

2. 一条日志的通用结构

```
Aug 16 08:30:01 ubuntu systemd[1]: Starting Daily apt upgrade and clean activities...
时间戳           主机名   程序名[进程号]   消息内容
```

规律：时间 + 谁写的 + 说什么。排错时按时间顺序看，就能还原"事发经过"。

2.5 日志都藏在哪儿？（去哪找线索）

位置	里面是什么
/var/log/syslog	系统总日志（系统里谁干了什么都记在这）
/var/log/auth.log	登录记录（谁登进来了、密码错了几次）
/var/log/nginx/access.log	网站访问日志（谁访问了哪个页面）
/var/log/nginx/error.log	网站错误日志
journalctl 命令	systemd 管理的服务日志（它是命令，不是文件）

心法：程序出问题，第一时间去它的日志文件里找线索——日志就是程序留给你的破案证据。

3. 日志级别体系（颜色代表严重程度）

级别	英文	颜色	含义
调试	DEBUG	灰	最啰嗦的细节，排错时看
信息	INFO	白	正常说话："我干了什么"
警告	WARNING	黄	"有点不对劲，但还能干活"
错误	ERROR	红	"出事了，这件事没干成"
严重	CRITICAL	红加粗	"大事故，我可能要挂了"

4. 读日志四步法（核心心法）

- ① 找到 **ERROR** 那一行（红色的就是案发现场）
- ② 从那一行往下读几行——错误的展开信息（Python 的 traceback 就是）
- ③ 从那一行往上读几行——出事前发生了什么（前因）
- ④ 还看不懂 → 复制报错原文去搜索引擎，前面加 ubuntu / linux / python

5. 报错的三种常见句式

✦ 句式①：X + 被动词（被动式）—— **Permission denied**（权限被拒绝）

✦ 句式②：cannot / can't + 动词 + 对象—— **cannot connect to host**（连不上主机）

✦ 句式③：No + 名词（缺东西）—— **No such file**（没这个文件）

A 组 · Linux 报错 (前 10 条必须熟读)

① Permission denied

逐词: **Permission** (权限) → **denied** (被拒绝, deny 的被动式)

整句: **权限被拒绝 = 你没资格动它**

为什么: 你要读/写/执行一个文件, 但你的身份权限不够

怎么办: **sudo** 借管理员身份; **chmod** 改权限; **chown** 换主人

② No such file or directory

逐词: **No** (没有) → **such** (这样的) → **file** (文件) → **or** (或者) → **directory** (目录)

整句: **没有这样的文件或目录 = 你指的路不存在**

为什么: 路径拼错了、文件名打错了、或你记错了位置

怎么办: **ls** 看看目录里到底有什么; **pwd** 确认自己在哪; Tab 自动补全防拼错

③ command not found

逐词: **command** (命令) → **not** (不) → **found** (被找到)

整句: **命令没找到 = 系统不认识你敲的东西**

为什么: ① 软件没装 ② 命令名拼错了 ③ 程序装了, 但它不在 PATH (系统的"命令搜索路径") 里, 系统不知道去哪找它

怎么办: 先 **which** 命令名 看装没装; 没装就 **sudo apt install** 软件名; 装了还找不到 → 重开一个终端 (或 **hash -r** 刷新命令缓存); 用 **echo \$PATH** 看搜索路径; 实在不行就用绝对路径运行, 如 **/usr/local/bin/nginx**

④ No space left on device

逐词: **No** (没有) → **space** (空间) → **left** (剩下) → **on device** (在设备上)

整句: **设备上没有剩余空间 = 磁盘满了**

为什么: 磁盘被写满了, 日志/文件/程序都写不进去了

怎么办: **df -h** 看哪块盘满; **du -sh /路径/*** 找谁占的空间; 删大文件、清 /tmp、清旧日志

⑤ Connection refused

逐词: **Connection** (连接) → **refused** (被拒绝)

整句: **连接被拒绝 = 门是关着的 (对方明确拒绝你)**

为什么: 对方的服务根本没启动; 或服务没监听你连的那个端口; 或防火墙直接挡了

怎么办: 确认服务启动没 (**systemctl status** 服务名); 用 **ss -tlnp** 看端口有没有人听; 查防火墙 **ufw status**

⑥ Connection timed out

逐词: **Connection** (连接) → **timed out** (超时了: time 时间 + out 用尽)

整句: **连接超时 = 敲了门, 等了很久没人应答**

为什么: 网络不通; 中间有防火墙悄悄丢包; 目标机器宕机; 路由不通

怎么办: **ping** 目标看通不通; **traceroute** 看断在哪一跳; 检查防火墙和安全组

💡 和 refused 的区别: refused = 对方明确说"不"; timed out = 对方连"不"都没说, 石沉大海。

⑦ Name or service not known

逐词: **Name** (名字, 指域名) → **service** (服务) → **not known** (不被知道)

整句: **域名解析不了 = 把网址翻译成 IP 的那一步失败了**

为什么: DNS 服务器连不上; 域名拼错; 本机 DNS 配置有问题

怎么办: `nslookup 域名` 测试解析; `ping 8.8.8.8` 确认网络通不通 (能 ping 通 IP 说明是 DNS 问题)

⑧ Address already in use

逐词: **Address** (地址, 指端口) → **already** (已经) → **in use** (在使用中)

整句: **端口已被占用 = 这个门牌号已经有人住了**

为什么: 你要启动的程序想监听 80 端口, 但另一个程序已经占着 80 了

怎么办: `ss -tlnp | grep 端口号` 找到占用的进程; kill 掉它或换一个端口启动

⑨ read-only file system

逐词: **read-only** (只读: read 读 + only 只有) → **file system** (文件系统)

整句: **文件系统是只读的 = 只能看不能写**

为什么: 磁盘硬件出问题系统会自动转只读保护; 或人为挂载成只读; U盘写保护

怎么办: 检查磁盘错误 (`fsck`); 重新挂载 `mount -o remount,rw /`; 硬件问题要换盘

⑩ Unable to lock the administration directory (/var/lib/dpkg/)

逐词: **Unable to** (无法) → **lock** (上锁) → **administration directory** (管理目录)

整句: **无法锁定管理目录 = 有另一个 apt 正在安装软件**

为什么: 两个安装命令同时跑, 或上次安装中断没清理锁

怎么办: 等另一个安装完成; 或删掉锁文件 `sudo rm /var/lib/dpkg/lock` 后重试

A 组续 • 其他常见 Linux 报错

⑪ Failed to fetch ... 404 Not Found

逐词: **Failed to fetch** (抓取失败: fetch = 拿取) → **404** (HTTP 状态码) → **Not Found** (没找到)

整句: **下载软件包失败, 404 = 服务器上没这个文件**

为什么: 软件源 (apt 的仓库) 失效、网络中断、或源地址配错了

怎么办: `sudo apt update` 刷新源; 换软件源 (清华/阿里镜像); 检查网络

⑫ E: Sub-process /usr/bin/dpkg returned an error code (1)

逐词: **Sub-process** (子进程) → **returned** (返回了) → **error code** (错误码)

整句: **安装过程中底层工具 dpkg 报错了, 返回错误码 1**

为什么: 软件包冲突、依赖坏了、或上次安装中断留下烂摊子

怎么办: `sudo dpkg --configure -a` 修复未完成的配置; 再 `sudo apt -f install` 修复依赖

⑬ Host key verification failed

逐词: **Host** (主机) → **key** (钥匙/指纹) → **verification** (验证) → **failed** (失败)

整句: **SSH 连接时主机指纹验证失败**

为什么: 第一次连某台机器时记住了它的"指纹", 之后指纹变了 (重装系统/换机器), SSH 怀疑你连的不是同一台, 怕被钓鱼

怎么办: 确认目标是安全的, 删掉旧指纹 `ssh-keygen -R 主机名/IP` 后重连

⑭ Permission denied (publickey)

逐词: **Permission denied** (权限被拒绝) → **publickey** (公钥)

整句: **SSH 免密登录失败: 服务器不认你的钥匙**

为什么: 你的公钥没加到服务器的 `authorized_keys` 里; 或钥匙权限太宽松 (必须是 600)

怎么办: 重新拷贝公钥 `ssh-copy-id user@host`; 检查 `chmod 600 ~/.ssh/authorized_keys`

⑮ Cannot connect to the Docker daemon. Is the docker daemon running?

逐词: **Cannot connect** (连不上) → **daemon** (守护进程: 后台服务) → **running** (运行中)

整句: **连不上 Docker 后台服务: 它没在运行**

为什么: Docker 服务没启动; 或当前用户没权限 (没加入 docker 组)

怎么办: `sudo systemctl start docker` 启动; `sudo usermod -aG docker 用户名` 加组后重新登录

⑯ port is already allocated

逐词: **port** (端口) → **already** (已经) → **allocated** (被分配)

整句: **Docker 端口已被占用** (和 ⑧ 同一个病)

怎么办: `docker ps` 看哪个容器占着; 改端口映射 `-p 8081:80`; 或 `ss -tlnp` 找到进程 kill 掉

⑰ Too many open files

逐词: **Too many** (太多) → **open** (打开着的) → **files** (文件)

整句: **打开的文件数超过上限**

为什么: Linux 每个进程能同时打开的文件数有限 (默认 1024), 程序忘了关文件或开了太多连接就会触发

怎么办: 临时调高 `ulimit -n 65535`; 长期改 `/etc/security/limits.conf`; 程序侧要记得 `close`

⑱ Segmentation fault (core dumped)

逐词: **Segmentation** (内存段) → **fault** (故障) → **core dumped** (核心转储: 把现场拍下来存了文件)

整句: **程序访问了不该访问的内存, 当场崩溃**

为什么: C/C++ 程序的野指针、越界访问等底层 bug; 内存条硬件故障也会

怎么办: 升级程序版本; 多次出现就查硬件 (memtest 测内存)

⑲ Killed (Out of memory)

逐词: **Killed** (被杀死) → **Out of memory** (内存耗尽)

整句: **系统内存不足, 内核的 OOM Killer 挑了个"最该杀的"进程杀掉**

为什么: 程序吃内存太多, 物理内存+交换空间全用光了

怎么办: `free -h` 看内存; `top` 找吃内存的进程; 给程序限内存或加 swap

B 组 · Python 报错（学 Python 后每天读几条）

💡 Python 报错的共同结构：**报错类型: 详细信息**，最后一行永远是最重要的——它直接告诉你**错误是什么**（类型+原因）；而具体的行号在它上面的 File "xxx.py", line 7 那一行里。读法：**从下往上读**。

B 组开篇 · 完整 traceback 逐行拆解（Python 报错的"全景图"）

Python 出错时打印的那一大段叫 **traceback**（追踪回溯）。看这个两层调用的完整例子：

```
Traceback (most recent call last):
  File "app.py", line 10, in <module>
    main()
  File "app.py", line 7, in main
    helper()
  File "app.py", line 3, in helper
    return 10 / 0
ZeroDivisionError: division by zero
```

逐行拆解（每行在说什么）：

原文	意思
Traceback (most recent call last):	trace(追踪)+back(回溯)=调用记录。整句："下面是事故的完整调用链， 最近的调用排在最后 "——越往下越接近案发现场
File "app.py", line 10, in <module>	File(文件)+line(行)+in(在)+<module>(模块顶层)。意思：案发文件 app.py，第 10 行，位置在顶层代码
main()	第 10 行的那句代码——它调用了函数 main（这是案发的第一层）
File "app.py", line 7, in main	进入 main 函数内部，追踪到第 7 行
helper()	main 里第 7 行调用了 helper（第二层）
File "app.py", line 3, in helper	进入 helper 函数内部，追踪到第 3 行
return 10 / 0	真正的凶手 ——这句代码除以了零
ZeroDivisionError: division by zero	报错类型+信息 = 除以零（最后一行 = 错误本质）

读法口诀（从下往上读）：① 最后一行 = 什么错 → ② 往上第一段代码 = 凶手是谁 → ③ 再往上 = 谁叫了它（调用链，帮你理清来龙去脉）。

① SyntaxError: invalid syntax

逐词：**Syntax**（语法）→ **Error**（错误）→ **invalid**（无效的）→ **syntax**（语法）

整句：**语法错误 = 你写的东西 Python 读不懂**

常见原因：冒号忘了、括号没配对、if/for 行尾没加：、用了中文标点

怎么办：看箭头 ^ 指的位置，检查它前面一行；最常漏的是冒号和右括号

② IndentationError: unexpected indent / expected an indented block

逐词：**Indentation**（缩进）→ **Error**（错误）→ **unexpected**（意外的）→ **indent**（缩进）；另一变体 **expected**（期望的）→ **block**（代码块）

整句：**缩进错了：Python 靠行首空格区分代码块**（冒号后面的下一行必须缩进，同一块的缩进必须对齐）

怎么办：冒号下一行加 4 个空格；同一层代码缩进对齐；别把空格和 Tab 混用

③ **NameError: name 'x' is not defined**

逐词: **Name** (名字) → **Error** (错误) → **is not defined** (没有被定义)

整句: **名字 x 不存在 = 这个变量你还没创建就用它**

常见原因: 变量名拼错了; 变量没赋值就用; 变量定义在函数里却想在函数外用

怎么办: 检查拼写; 确认赋值语句在用它之前执行了

④ **TypeError: can only concatenate str (not "int") to str**

逐词: **Type** (类型) → **Error** (错误) → **concatenate** (拼接) → **str** (字符串) → **int** (整数)

整句: **类型错误: 字符串只能拼字符串, 你拿字符串去拼整数了**

例: "年龄: " + 19 就报这个错; 正确是 "年龄: " + str(19)

怎么办: 用 str()/int()/float() 转换类型再运算; print 里改用逗号或 f-string

⑤ **ValueError: invalid literal for int() with base 10: 'abc'**

逐词: **Value** (值) → **Error** (错误) → **invalid literal** (无效的字面量) → **base 10** (十进制)

整句: **值错误: 'abc' 不是数字, int() 没法把它转成整数**

怎么办: 转换前先确认内容是数字; 用 try/except 包住转换

⑥ **KeyError: 'age'**

逐词: **Key** (键) → **Error** (错误)

整句: **字典里没有 'age' 这个键, 你查了个不存在的名字**

怎么办: 用 d.get("age") 代替 d["age"] (get 查不到返回 None 不报错)

⑦ **IndexError: list index out of range**

逐词: **Index** (索引) → **Error** (错误) → **out of range** (超出范围)

整句: **列表索引越界 = 你要第 10 个, 但列表只有 5 个**

为什么: 列表索引从 0 开始数! 长度 5 的列表, 最后一个的索引是 4, 访问 [5] 就炸

怎么办: 用 len() 先量长度; 循环里用 for x in list 而不是手数索引

⑧ **AttributeError: 'list' object has no attribute 'keys'**

逐词: **Attribute** (属性) → **object** (对象) → **has no attribute** (没有这个属性)

整句: **属性错误: 列表没有 keys 这个方法 (你把它当字典用了)**

为什么: 对象类型搞错了——keys() 是字典的方法, 列表没有

怎么办: 用 type(变量) 确认类型; 别把 list 和 dict 混着用

⑨ **ModuleNotFoundError: No module named 'requests'**

逐词: **Module** (模块) → **Not Found** (没找到) → **named** (名字叫)

整句: **找不到 requests 这个模块 = 这个库你没安装**

怎么办: **pip install requests**; 装完还是报错就检查是不是装到了别的 Python 版本里

⑩ **ImportError: cannot import name 'xxx' from 'yyy'**

逐词: **Import** (导入) → **cannot import** (无法导入) → **from** (从...)

整句: **导入错误: yyy 模块里没有 xxx 这个东西**

常见原因: 名字拼错; 模块版本太旧没有这个函数; 循环导入 (两个文件互相 import)

怎么办: `dir(yyy)` 看看模块里到底有什么; 升级模块版本

⑪ **FileNotFoundError: [Errno 2] No such file or directory: 'data.json'**

逐词: **File** (文件) → **Not Found** (没找到) → **Errno 2** (系统错误编号 2)

整句: **Python 找不到 data.json 这个文件**

为什么: 文件不在当前目录; 路径写错; 文件名拼错

怎么办: `pwd` 和 `ls` 确认文件在不在脚本所在目录; 用绝对路径最稳妥

⑫ **PermissionError: [Errno 13] Permission denied: 'file.txt'**

逐词: **Permission** (权限) → **Error** (错误) → **denied** (被拒绝)

整句: **Python 打开文件时被系统拒绝 (没权限)** ——和 Linux 的 Permission denied 是同一个病

怎么办: `chmod` 改权限; `sudo` 运行; 别去碰系统目录里的文件 (`/etc`、`/root`)

⑬ **ZeroDivisionError: division by zero**

逐词: **Zero** (零) → **Division** (除法) → **by** (被) → **zero** (零)

整句: **除以零了 = 数学上不允许的操作**

怎么办: 除法前判断分母是不是 0: `if b != 0: a / b`

⑭ **EOFError: EOF when reading a line**

逐词: **EOF** (End Of File: 文件结束) → **Error** (错误) → **when reading** (在读取时)

整句: **还没读完就结束了 = `input()` 等输入, 但输入流没了**

为什么: `input()` 在没人输入的环境里 (比如在线判题平台) 就会遇到

怎么办: 给 `input` 提供数据; 或不用 `input` 改用参数传入

⑮ **UnboundLocalError: local variable 'x' referenced before assignment**

逐词: **local variable** (局部变量) → **referenced** (被引用) → **before assignment** (在赋值之前)

整句: **函数里用了还没赋值的局部变量 (赋值前就读取了)**

为什么: 函数内给 `x` 赋值 → Python 把 `x` 当局部变量 → 但赋值语句在读取语句后面

怎么办: 把赋值挪到读取前面; 要改全局变量就在函数里加 `global x`

⑯ **RecursionError: maximum recursion depth exceeded**

逐词: **Recursion** (递归) → **maximum** (最大的) → **depth** (深度) → **exceeded** (被超过)

整句: **递归太深 = 函数自己叫自己叫个没完 (约1000层后 Python 强制叫停)**

为什么: 递归函数忘了写"停下来的条件" (基准条件)

怎么办: 检查递归的出口条件写没写、写得对不对

⑩ JSONDecodeError: Expecting value: line 1 column 1 (char 0)

逐词: **Expecting value** (期待一个值) → **line 1 column 1** (第 1 行第 1 列)

整句: **JSON 解析失败: 想解析的内容根本不是 JSON (第1个字符就不是)**

为什么: API 返回了 HTML 报错页/空内容, 你却当 JSON 解析

怎么办: 先把返回内容 print 出来看看是什么; 确认接口地址和返回格式

⑪ OSError: [Errno 98] Address already in use

逐词: **Address** (地址) → **already** (已经) → **in use** (在使用中)

整句: **端口被占用**——Python 起服务 (如 Flask) 时和 Linux 报错 ⑧ 同一个病

怎么办: 换端口; `ss -tlnp | grep 端口` 找到占用者 kill 掉

⑫ MemoryError

逐词: **Memory** (内存) → **Error** (错误)

整句: **内存不足 = Python 程序吃光内存**

为什么: 一次读入超大文件; 死循环里不停 append 列表; 数据处理量爆炸

怎么办: 大文件用流式读取 (一行行读); 检查死循环; 精简数据结构

⑬ KeyboardInterrupt

逐词: **Keyboard** (键盘) → **Interrupt** (打断)

整句: **你按了 Ctrl+C 把程序打断了 (这不是 bug, 是你干的)**

怎么办: 不用管, 程序是正常的, 你想重启就再运行一次

尾页 · 排错心法 (遇到没见过的报错怎么办)

① **别慌, 报错是线索不是判决**——程序在告诉你它为什么死, 不是骂你

② **读最后一行**——Python 报错最后一行是案发第一现场, 前面的是"谁叫了谁"的追踪链

③ **复制原文去搜索**——前面加 python / ubuntu, 全世界和你踩过同一个坑的人成千上万

④ **一次只改一个变量**——排错时改一处试一次, 别一次性改五处 (改五处, 好了你也不知道是哪处起作用)

⑤ **解决后记笔记**——"什么报错 + 什么原因 + 怎么解决"写进 GitHub, 这以后就是你的《运维踩坑宝典》